



PROJET DE VISION PAR ORDINATEUR

Détection de maladies foliaires par YOLO

Fine-tuning de YOLO11n sur le dataset FieldPlant (Roboflow Universe)

Rapport d'expérimentation

Réalisé par : KOYAKPEGON Josué

Projet CV — Détection de maladies foliaires (YOLO)

Durée du projet : 10 jours — Septembre 2026

Environnement d'entraînement : CPU uniquement (Intel Core i7-8650U), sans GPU

Table des matières

1. Introduction et objectifs
2. Jeu de données
3. Environnement de travail et contrainte matérielle
4. Méthodologie d'entraînement
5. Résultats
6. Discussion : compromis précision / vitesse
7. Limites et perspectives
8. Conclusion

1. Introduction et objectifs

Ce rapport présente le travail réalisé dans le cadre du projet de vision par ordinateur portant sur la détection de maladies foliaires à l'aide d'un modèle YOLO. L'objectif était de concevoir un pipeline complet : préparation et vérification d'un jeu de données annoté en bounding boxes, entraînement d'un modèle de détection d'objets, puis évaluation de ses performances selon les métriques standard du domaine (mAP, précision, rappel, F1-score).

Le but final est d'obtenir un modèle capable de localiser une lésion ou un symptôme sur une feuille et de l'associer à une maladie, dans des conditions proches du terrain (arrière-plans complexes, éclairage variable, plusieurs cultures). Un tel système peut, à terme, servir d'outil d'aide au diagnostic phytosanitaire précoce, notamment pour des cultures comme le manioc, le maïs et la tomate.

Je précise dès l'introduction un point qui a fortement conditionné mes choix méthodologiques tout au long du projet : je n'ai travaillé que sur un ordinateur portable équipé d'un CPU (Intel Core i7-8650U), sans carte graphique dédiée. Cette contrainte, détaillée dans la section 3, explique notamment pourquoi j'ai procédé en deux temps (un test rapide sur 3 époques, puis un entraînement principal sur 30 époques) plutôt que de lancer directement un entraînement long.

2. Jeu de données

2.1 Présentation du dataset FieldPlant

Le jeu de données utilisé est FieldPlant, disponible sur Roboflow Universe. Il s'agit d'un dataset public déjà annoté en bounding boxes par des experts en pathologie végétale, avec la particularité d'avoir été constitué à partir de photos prises en conditions réelles de champ : arrière-plans encombrés (terre, autres plantes, ombres), plusieurs cultures mélangées et une luminosité naturelle non contrôlée. C'était pour moi un critère important : autant s'entraîner directement sur des images qui ressemblent à ce qu'un agriculteur ou un technicien prendrait sur le terrain avec son téléphone, plutôt que sur des feuilles isolées sur fond neutre.

Le dataset comprend 27 classes couvrant trois cultures (manioc, maïs, tomate), avec pour chacune un état sain et plusieurs maladies ou symptômes distincts. Le tableau ci-dessous récapitule les classes telles que définies dans le fichier data.yaml.

ID	Culture / classe	ID	Culture / classe
0	Flétrissement bactérien du manioc	14	Stries du maïs
1	Tache brune des feuilles du manioc	15	Rayures du maïs
2	Manioc sain	16	Décoloration violette du maïs
3	Mosaïque du manioc	17	Taches jaunes du maïs
4	Pourriture des racines du manioc	18	Jaunissement du maïs
5	Taches brunes du maïs	19	Brûlure des feuilles du maïs
6	Charbon du maïs	20	Rouille des feuilles du maïs
7	Tache chlorotique des feuilles du maïs	21	Taches brunes de la tomate

ID	Culture / classe	ID	Culture / classe
8	Tache grise des feuilles du maïs	22	Flétrissement bactérien de la tomate
9	Maïs sain	23	Brûlure des feuilles de la tomate
10	Dégâts d'insectes sur le maïs	24	Tomate saine
11	Mildiou du maïs	25	Virus de la mosaïque des feuilles de tomate
12	Décoloration violette du maïs	26	Virus du jaunissement des feuilles de tomate
13	Charbon du maïs		

À noter : les identifiants 6/13 (« Charbon du maïs ») et 12/16 (« Décoloration violette du maïs ») correspondent à des libellés dupliqués dans le fichier de configuration d'origine. Je n'ai pas corrigé cette redondance pour rester fidèle à l'annotation fournie par le dataset, mais cela mérite d'être signalé pour la suite (voir section 7).

2.2 Vérification et répartition du dataset

Avant tout entraînement, j'ai écrit un script de vérification qui contrôle, pour chaque split (train/valid/test), que chaque image possède bien un fichier d'annotation associé, que chaque ligne YOLO contient exactement 5 valeurs, que les identifiants de classe sont compris entre 0 et 26, et que les coordonnées normalisées sont bien comprises entre 0 et 1. J'ai également vérifié l'absence de chevauchement entre les trois ensembles (aucune image présente dans deux splits à la fois).

Résultat : le dataset est entièrement valide, sans anomalie détectée.

Ensemble	Images	Labels	Images sans annotation	Statut
Train	3 609	3 609	0	Valide
Validation	773	773	0	Valide
Test	774	774	0	Valide
Total (unique)	5 156	—	—	Aucun chevauchement

2.3 Répartition des classes

En comptant le nombre d'annotations par classe sur l'ensemble d'entraînement, on observe un déséquilibre assez marqué : trois classes (taches brunes de la tomate, mosaïque du manioc, brûlure des feuilles du maïs) concentrent à elles seules plus de la moitié des annotations, alors qu'à l'inverse, plusieurs classes rares (charbon du maïs, décoloration violette du maïs, flétrissement bactérien de la tomate) comptent moins de 10 exemples. Cette répartition inégale est un point que je retrouve plus loin dans l'analyse des résultats : sans surprise, ce sont globalement les classes les mieux représentées qui obtiennent les meilleurs scores.

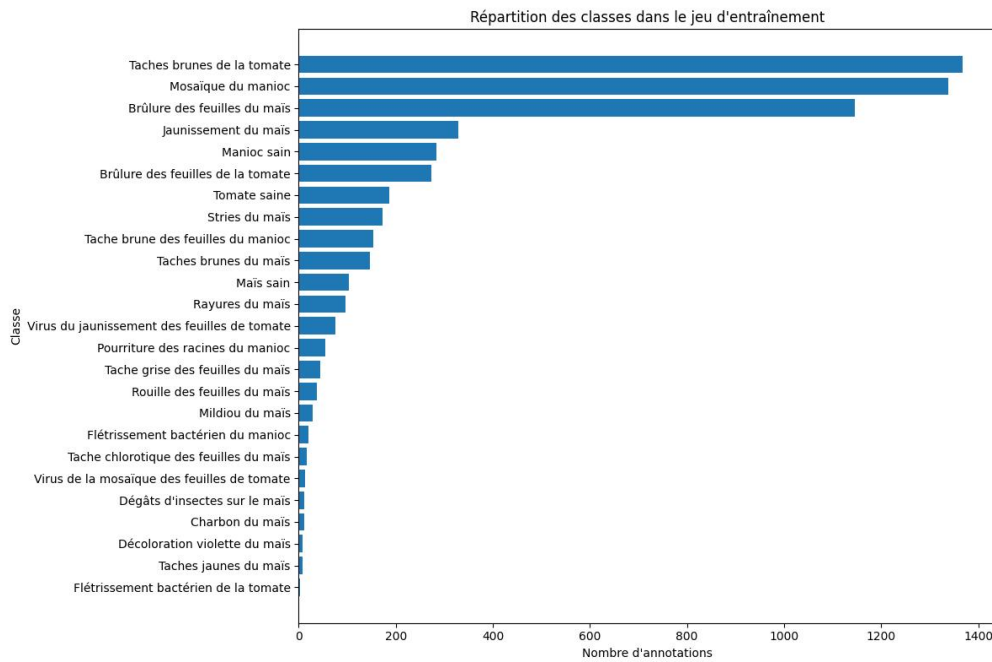


Figure 1 — Répartition des annotations par classe dans le jeu d'entraînement

2.4 Exemples d'annotations

Pour vérifier visuellement la cohérence des annotations avant de lancer l'entraînement, j'ai affiché aléatoirement quelques images avec leurs bounding boxes superposées. On voit bien, sur l'exemple ci-dessous, la difficulté du dataset : plusieurs feuilles se chevauchent, l'arrière-plan est chargé (terre, autres plantes), et les boîtes sont parfois assez rapprochées les unes des autres.

Manioc_Mosaïque_-287_.jpg.rf.5e7dbf027eec32f3c7e6f5090595a869.jpg

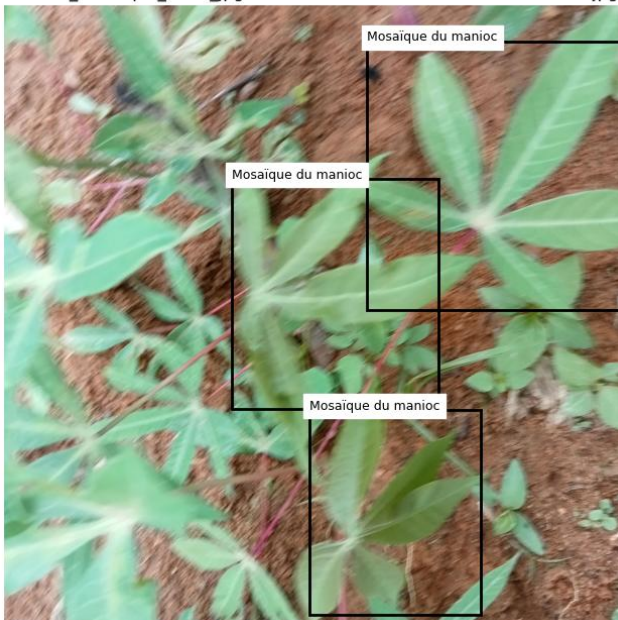


Figure 2 — Exemple d'image annotée : mosaïque du manioc, plusieurs occurrences sur la même feuille

3. Environnement de travail et contrainte matérielle

C'est probablement le point le plus déterminant de tout ce projet, donc je préfère l'expliquer clairement plutôt que de le glisser en note de bas de page : je n'ai eu accès qu'à un ordinateur portable classique (Intel Core i7-8650U, 1.90 GHz), sans GPU compatible CUDA. La vérification de l'environnement au tout début du notebook le confirme sans ambiguïté.

Élément	Valeur
Processeur	Intel Core i7-8650U @ 1.90 GHz
GPU CUDA disponible	Non
Framework	PyTorch 2.14.0+cpu
Dispositif d'entraînement	device="cpu"
Bibliothèque de détection	Ultralytics YOLO (YOLO11n)

Concrètement, l'absence de GPU change complètement l'échelle de temps d'un entraînement YOLO. À titre indicatif, une époque sur ce dataset (3 609 images, batch=4, imgsz=640) a pris entre 50 minutes et un peu plus d'une heure sur mon CPU, contre quelques dizaines de secondes seulement sur une carte GPU récente. Cette contrainte a orienté plusieurs décisions :

- Choix du modèle le plus léger de la famille YOLO11 (YOLO11n, ~2,6 millions de paramètres) plutôt qu'une variante « small » ou « medium », pour limiter le temps de calcul par image.
- Une taille de batch réduite (batch=4) et 0 worker de chargement de données (workers=0), pour rester dans les limites de RAM et de CPU disponibles.
- Une stratégie en deux phases (un test court, puis un entraînement plus long) plutôt qu'un unique entraînement long lancé à l'aveugle, pour éviter de perdre plusieurs jours sur une configuration buguée.
- Je n'ai pas pu comparer nano vs small comme suggéré dans le plan de projet, faute de temps de calcul disponible sur les 10 jours impartis — un point que je reprends dans les limites (section 7).

4. Méthodologie d'entraînement

4.1 Modèle et configuration générale

Le modèle utilisé est YOLO11n (« nano »), initialisé avec les poids pré-entraînés yolo11n.pt fournis par Ultralytics, puis fine-tuné sur le dataset FieldPlant. La configuration commune aux deux phases d'entraînement est la suivante : résolution d'entrée imgsz=640, batch=4, optimiseur automatique (optimizer="auto"), graine aléatoire fixée à 42 pour la reproductibilité, et device="cpu".

4.2 Phase 1 — Test rapide sur 3 époques

Avant de me lancer dans un entraînement long, j'ai d'abord fait tourner le pipeline sur seulement 3 époques (patience=2). L'idée n'était pas d'obtenir un bon modèle à ce stade, mais simplement de m'assurer que tout fonctionnait correctement de bout en bout — chargement du data.yaml, calcul des pertes, sauvegarde des checkpoints, génération des courbes — avant d'immobiliser la machine pendant plusieurs heures pour rien. Avec le recul, cette étape s'est révélée indispensable : sur CPU, une erreur de configuration découverte seulement après 20 heures d'entraînement aurait été un vrai gâchis de temps sur un projet de 10 jours.

Ce test de 3 époques a représenté environ 2 h 39 min de calcul (voir tableau ci-dessous), soit un peu moins de 53 minutes par époque en moyenne.

Époque	Precision	Recall	mAP@0.5	mAP@0.5:0.95
1	0.6916	0.1631	0.1221	0.0910
2	0.6732	0.1829	0.2130	0.1509
3	0.6177	0.2224	0.2409	0.1786

Les résultats restent logiquement faibles ($\text{mAP@0.5} \approx 0,24$ à la 3^e époque) puisque le modèle n'a eu que très peu de temps pour apprendre, mais la tendance est clairement à la hausse sur les trois métriques de détection, et surtout aucune erreur bloquante n'est apparue. J'ai donc pu enchaîner sereinement sur l'entraînement principal.

4.3 Phase 2 — Entraînement principal sur 30 époques

Une fois le pipeline validé, j'ai relancé un entraînement complet à partir des poids pré-entraînés yolo11n.pt, cette fois sur 30 époques avec une patience de 8 (arrêt anticipé si aucune amélioration pendant 8 époques consécutives). Cet entraînement n'a jamais déclenché l'arrêt anticipé : la métrique mAP@0.5:0.95 continuait de progresser jusqu'à la toute dernière époque, ce qui suggère que le modèle aurait probablement encore progressé avec davantage d'époques — mais le temps de calcul disponible sur CPU ne me permettait pas d'aller plus loin dans le cadre des 10 jours du projet.

Cet entraînement principal a duré environ 34 h 25 min sur CPU, soit un peu plus d'une journée et demie de calcul continu. C'est de loin le poste qui a consommé le plus de temps sur l'ensemble du projet, et il a fallu planifier son lancement en avance (nuit + journée) pour ne pas bloquer l'avancement du reste du travail.

5. Résultats

5.1 Courbes d'apprentissage (30 époques)

La figure ci-dessous regroupe l'ensemble des courbes générées automatiquement par Ultralytics : pertes d'entraînement et de validation (box, classification, dfl), ainsi que precision, recall, mAP@0.5 et mAP@0.5:0.95 au fil des époques. On observe une décroissance régulière des pertes et une progression continue des métriques de détection, sans signe net de sur-apprentissage à ce stade — les courbes de validation suivent d'assez près celles d'entraînement.

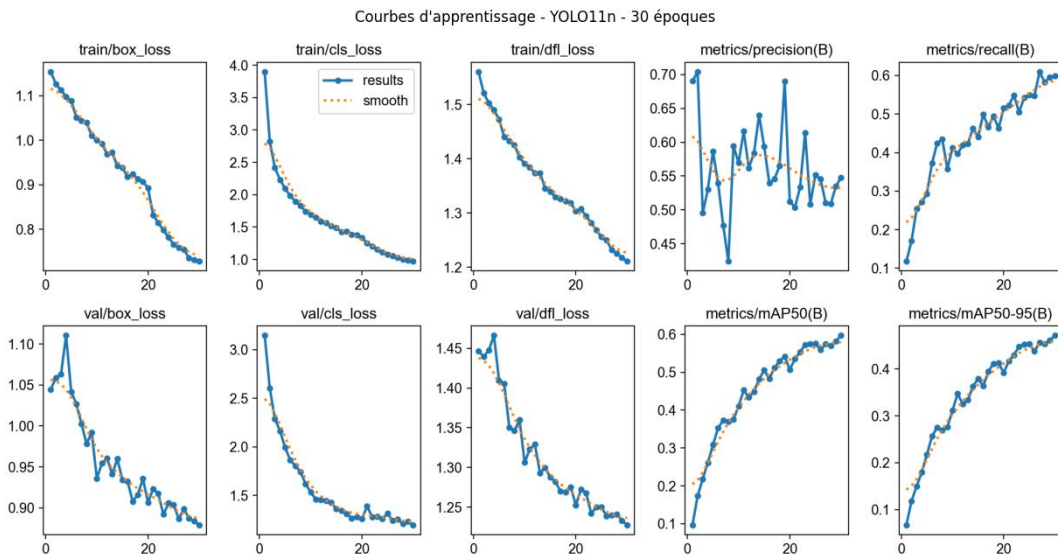


Figure 3 — Courbes d'apprentissage complètes, YOLO11n, 30 époques (CPU)

Le tableau suivant compare les trois moments clés de l'entraînement : la fin du test rapide (3 époques), la meilleure époque de l'entraînement principal (30 époques), et l'évaluation finale indépendante sur le jeu de test (section 5.2). La progression entre le test rapide et l'entraînement complet est nette sur toutes les métriques, ce qui confirme que le temps d'entraînement supplémentaire — malgré la contrainte CPU — a été rentable.

Métrique	Test rapide (3 ép.)	Entraînement principal (30 ép., meilleure ép.)	Évaluation finale (test)
Precision	0.618	0.548	0.493
Recall	0.222	0.599	0.592
mAP@0.5	0.241	0.597	0.559
mAP@0.5:0.95	0.179	0.471	0.436

On remarque que la precision baisse légèrement entre l'entraînement (0.548) et l'évaluation finale sur le jeu de test totalement indépendant (0.493), pendant que le recall progresse (de 0.599 à 0.592, quasi stable). C'est un comportement assez classique : le modèle généralise raisonnablement bien, avec une légère baisse de performance sur des images qu'il n'a jamais vues, ce qui reste cohérent et rassurant plutôt qu'alarmant.

5.2 Évaluation finale sur le jeu de test

Le meilleur modèle sauvegardé (best.pt, poids correspondant à la meilleure époque selon mAP@0.5:0.95) a ensuite été évalué sur le jeu de test, totalement indépendant du train et de la validation (774 images, 1 353 instances annotées). Les résultats globaux et par classe sont détaillés ci-dessous.

Ensemble	Images	Instances	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Toutes classes	774	1 353	0.493	0.592	0.559	0.436

Détail par classe (seules les classes présentes dans le jeu de test sont affichées ; certaines classes très

rares du train, comme la « décoloration violette du maïs », n'ont aucune occurrence dans le split test) :

Classe	Img.	Inst.	P	R	mAP@0.5	mAP@0.5:0.95
Flétrissement bactérien du manioc	3	3	0.409	0.273	0.299	0.272
Tache brune des feuilles du manioc	23	24	0.478	0.667	0.595	0.559
Manioc sain	38	61	0.490	0.902	0.781	0.626
Mosaïque du manioc	158	286	0.723	0.804	0.842	0.622
Pourriture des racines du manioc	12	13	0.677	0.692	0.645	0.532
Taches brunes du maïs	23	28	0.376	0.571	0.457	0.232
Charbon du maïs	1	1	0.000	0.000	0.000	0.000
Tache chlorotique des feuilles du maïs	4	4	0.492	0.500	0.491	0.456
Tache grise des feuilles du maïs	2	4	0.322	1.000	0.836	0.646
Maïs sain	15	27	0.458	0.667	0.456	0.365
Dégâts d'insectes sur le maïs	1	1	1.000	0.000	0.199	0.199
Mildiou du maïs	7	7	0.438	0.143	0.473	0.312
Charbon du maïs (2)	1	1	0.000	0.000	0.000	0.000
Stries du maïs	40	51	0.480	0.843	0.733	0.520
Rayures du maïs	25	43	0.323	0.651	0.603	0.365
Taches jaunes du maïs	2	2	0.501	1.000	0.663	0.663
Jaunissement du maïs	44	80	0.576	0.713	0.632	0.354
Brûlure des feuilles du maïs	167	264	0.643	0.758	0.740	0.500
Rouille des feuilles du maïs	3	3	0.210	0.667	0.452	0.451
Taches brunes de la tomate	143	316	0.764	0.862	0.897	0.744
Brûlure des feuilles de la tomate	57	61	0.698	0.945	0.924	0.651
Tomate saine	30	54	0.218	0.556	0.374	0.264
Virus mosaïque feuilles tomate	2	3	1.000	0.000	0.426	0.366
Virus jaunissement feuilles tomate	10	16	0.545	1.000	0.904	0.755

Deux observations me semblent importantes ici. D'abord, sans surprise compte tenu de la section 2.3, les classes les mieux détectées (mosaïque du manioc, taches brunes de la tomate, brûlure des feuilles de la tomate, virus du jaunissement des feuilles de tomate) sont globalement celles qui disposaient du plus grand nombre d'exemples d'entraînement, ou au moins d'un nombre raisonnable. Ensuite, les classes avec un seul exemple dans le jeu de test (charbon du maïs, dégâts d'insectes sur le maïs) donnent des scores peu fiables statistiquement : un mAP de 0 ou de 1 sur une seule instance ne veut pas dire grand-chose en soi, il faudrait beaucoup plus de données pour ces classes rares avant de tirer une vraie conclusion.

5.3 Analyse qualitative

Au-delà des métriques chiffrées, j'ai regardé un certain nombre de prédictions visuellement. Les bonnes détections concernent en priorité les symptômes larges et bien contrastés (mosaïque du manioc, brûlures étendues sur les feuilles de tomate ou de maïs), où les bounding boxes prédites collent bien aux zones annotées manuellement. À l'inverse, les erreurs les plus fréquentes se concentrent sur deux

cas de figure : les symptômes très petits ou en tout début d'apparition (par exemple certaines taches naissantes, difficiles à distinguer même à l'œil nu sur l'image), et la confusion entre des maladies qui produisent des marques visuellement proches sur une même culture — par exemple entre différentes taches brunes/grises du maïs. Ce type de confusion inter-classes n'est pas surprenant vu la ressemblance visuelle réelle entre certains symptômes, et il rejoint typiquement les difficultés déjà documentées sur ce genre de dataset agricole de terrain.

IMG_3493.JPG.rf.8b47c7639ee71ece1468afe250807f4e.jpg

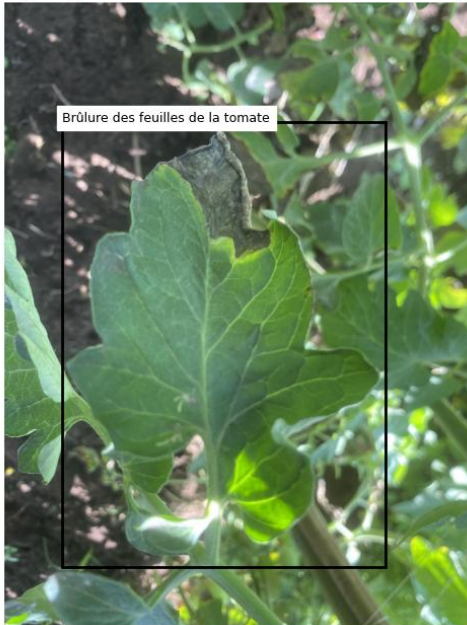


Figure 4 — Exemple d'annotation de référence utilisée pour comparer visuellement les prédictions du modèle

6. Discussion : compromis précision / vitesse

Le modèle YOLO11n retenu est volontairement le plus léger de sa famille (2 587 417 paramètres, 6,4 GFLOPs, 100 couches après fusion). Lors de l'évaluation finale, le temps mesuré était d'environ 114 ms d'inférence par image sur CPU (auxquels s'ajoutent 2,9 ms de pré-traitement et 2,5 ms de post-traitement, soit un total proche de 119 ms/image). Cela correspond à un débit d'environ 8 images par seconde sur mon processeur — largement suffisant pour un usage de diagnostic différé (on prend une photo, on l'envoie, on attend le résultat quelques centaines de millisecondes), mais clairement insuffisant pour de la détection vidéo temps réel sur ce même matériel.

Ce choix de modèle nano n'était pas neutre : une variante YOLO11s aurait probablement amélioré la précision de quelques points, au prix d'un temps d'entraînement et d'inférence significativement plus long. Sur GPU, ce compromis se discute facilement en testant les deux variantes ; sur CPU et avec seulement 10 jours de projet, je n'ai tout simplement pas eu la marge de temps nécessaire pour le faire sérieusement (voir section suivante).

7. Limites et perspectives

- Absence de GPU : c'est de loin la limite la plus contraignante du projet. Elle a réduit le nombre d'expériences possibles à une seule configuration complète (YOLO11n, 30 époques), sans possibilité réaliste de comparer plusieurs architectures ou hyperparamètres dans le temps imparti.
- Déséquilibre des classes : certaines maladies rares (moins de 10 annotations dans le train) restent mal détectées, voire non détectables de façon fiable. Un futur travail pourrait s'appuyer sur du sur-échantillonnage, de la data augmentation ciblée sur ces classes, ou la collecte de données supplémentaires.
- Classes dupliquées dans data.yaml (« Charbon du maïs » et « Décoloration violette du maïs » apparaissant deux fois sous des identifiants différents) : cela dilue artificiellement les exemples disponibles pour ces symptômes et mériterait d'être nettoyé avant un futur entraînement.
- Le mAP@0.5:0.95 continuait de progresser à la 30^e époque sans signe de plateau : un entraînement plus long (50-100 époques) apporterait probablement un gain de performance supplémentaire, mais représenterait plusieurs jours de calcul CPU additionnels.
- Comparaison nano vs small suggérée dans le plan de projet initial : non réalisée faute de temps de calcul, mais identifiée comme prochaine étape naturelle si un GPU devient disponible.

8. Conclusion

Ce projet m'a permis de mettre en place un pipeline complet de détection de maladies foliaires avec YOLO, de la vérification du dataset jusqu'à l'évaluation finale sur un jeu de test indépendant. Le modèle obtenu (YOLO11n fine-tuné, 30 époques) atteint un mAP@0.5 de 0,559 et un mAP@0.5:0.95 de 0,436 sur le jeu de test, avec des performances très hétérogènes selon les classes — bonnes sur les maladies bien représentées et communes à plusieurs cultures, plus fragiles sur les symptômes rares ou visuellement ambigus.

Le point que je retiens le plus de cette expérience, c'est à quel point la contrainte matérielle (CPU seul, pas de GPU) a pesé sur la méthodologie elle-même : le choix du modèle le plus léger, la stratégie de test avant entraînement complet, et le renoncement à certaines comparaisons pourtant prévues initialement en sont la conséquence directe. Avec un GPU et davantage de temps, les prochaines étapes logiques seraient d'allonger l'entraînement, de rééquilibrer les classes rares, et de comparer plusieurs tailles de modèle YOLO11 pour affiner le compromis précision/vitesse.

Lien Github : <https://github.com/koyakpegonjo-cell/detection-maladies-foliaires>